

CS315 Project Presentation

MirrorShield: Cross-Architecture Linux Malware Analysis via Lightweight Emulation and LLMs

Team Member:
Zhang Zihan
Lin Wenqi
Wang Hongyi

Instructor:
Zhang Fengwei



Special Funds for the Cultivation of Guangdong College Students' Scientific
and Technological Innovation.pdjh2026c11106

The Problem: A Surge in Fragmented Linux Malware

The proliferation of IoT has shifted the threat landscape, creating a massive, diverse, and challenging new battlefield for cybersecurity.

1.5 Billion+
malware samples
projected by 2025.

Key Trend

Primary threat focus has shifted from Windows to Linux, driven by the explosion of IoT and embedded systems.

The Challenge of Fragmentation

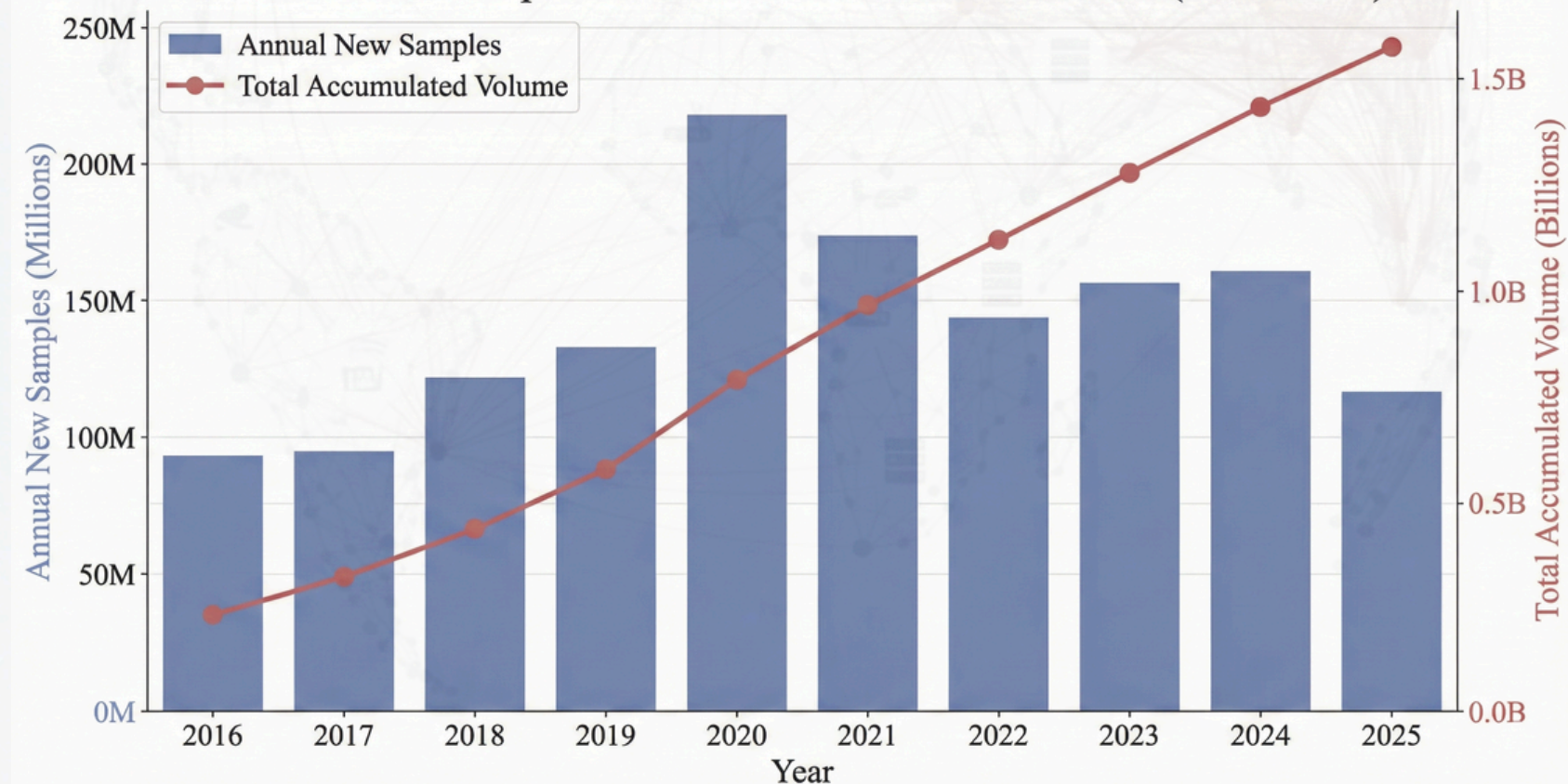
Unlike standardized Windows environments, Linux malware targets a wide diversity of CPU architectures (e.g., x86, MIPS, ARM) and has complex runtime dependencies.

Recent Attack

DDoS attack targeting DeepSeek orchestrated by Linux-based botnets like HailBot and RapperBot.

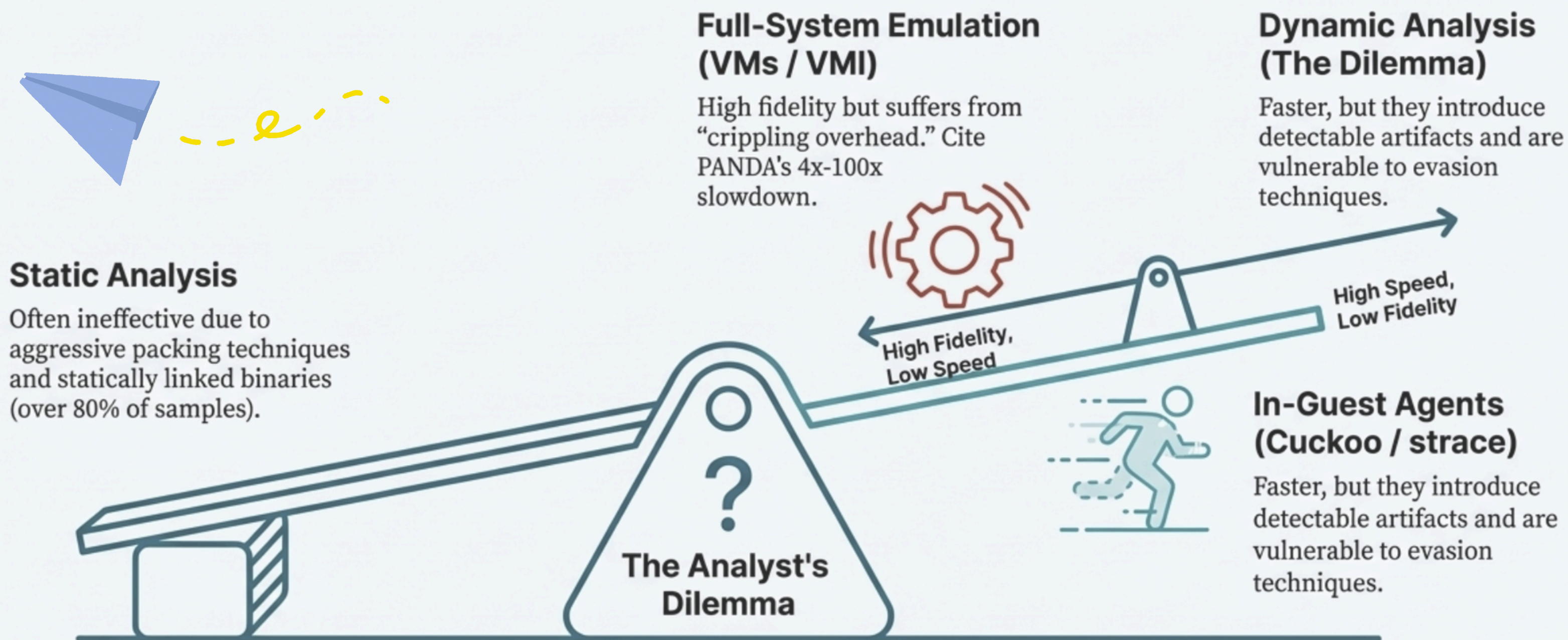


Annual New Samples vs. Total Accumulated Malware (2016-2025)



The Core Challenge: Why Existing Tools Falter

Current analysis methods force a trade-off between performance and fidelity, and neither approach is well-suited for the fragmented Linux ecosystem.





We have identified **three critical challenges** in the current field of dynamic malware analysis:

1. **Anti-debugging and anti-virtualization techniques.** Malware actively employs evasion tactics to avoid detection.
2. **The overwhelming volume of malware.** Security analysts are unable to manually debug and inspect each sample individually, making fast, scalable, and automated detection a pressing issue (the hard topic highlighted at the ACSAC 2025 conference).
3. The growing need for **cross-architecture analysis** driven by the expansion of IoT devices.
4. AI/ML to describe models (like SVMs or Neural Networks) where you put data in and get a label out (e.g., "Malicious"), **but you don't know why.**

Therefore, our goal is to develop an **automated malware analysis sandbox** that is **scalable, rapid, and supports multiple architectures** enabling semantic interpretation and detailed forensic reconstruction of malicious behaviors.

01

Related Work

Work summary



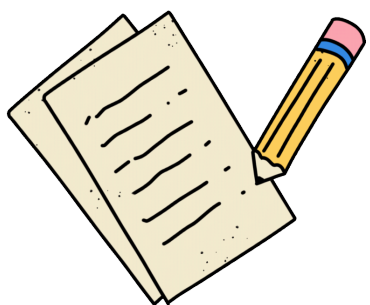
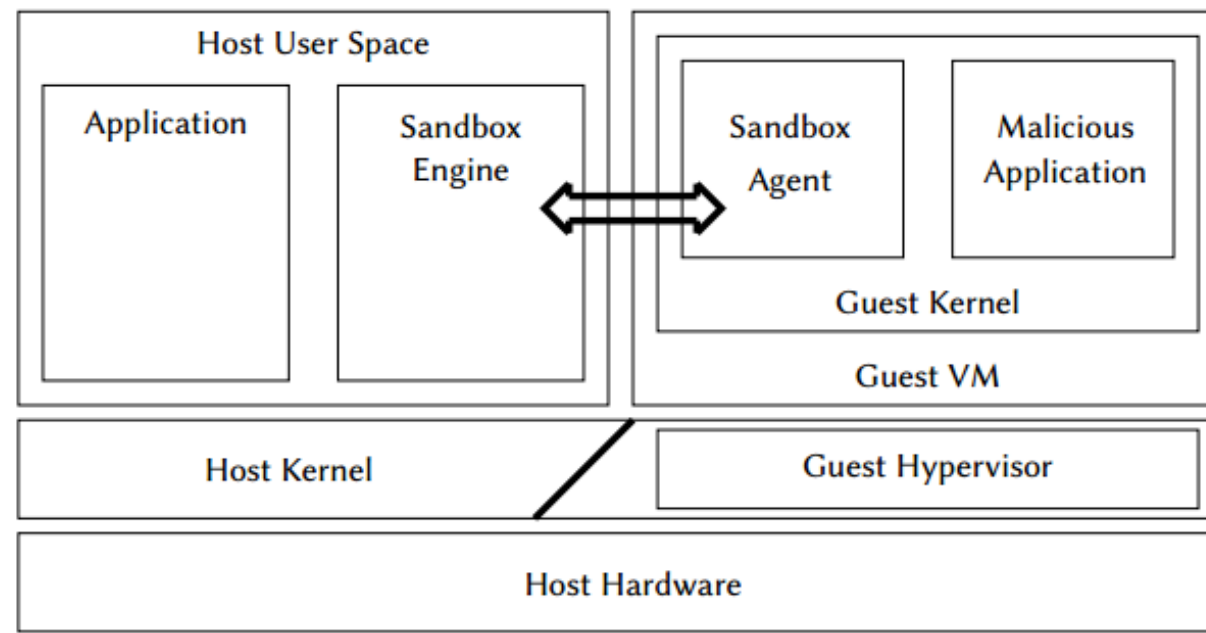


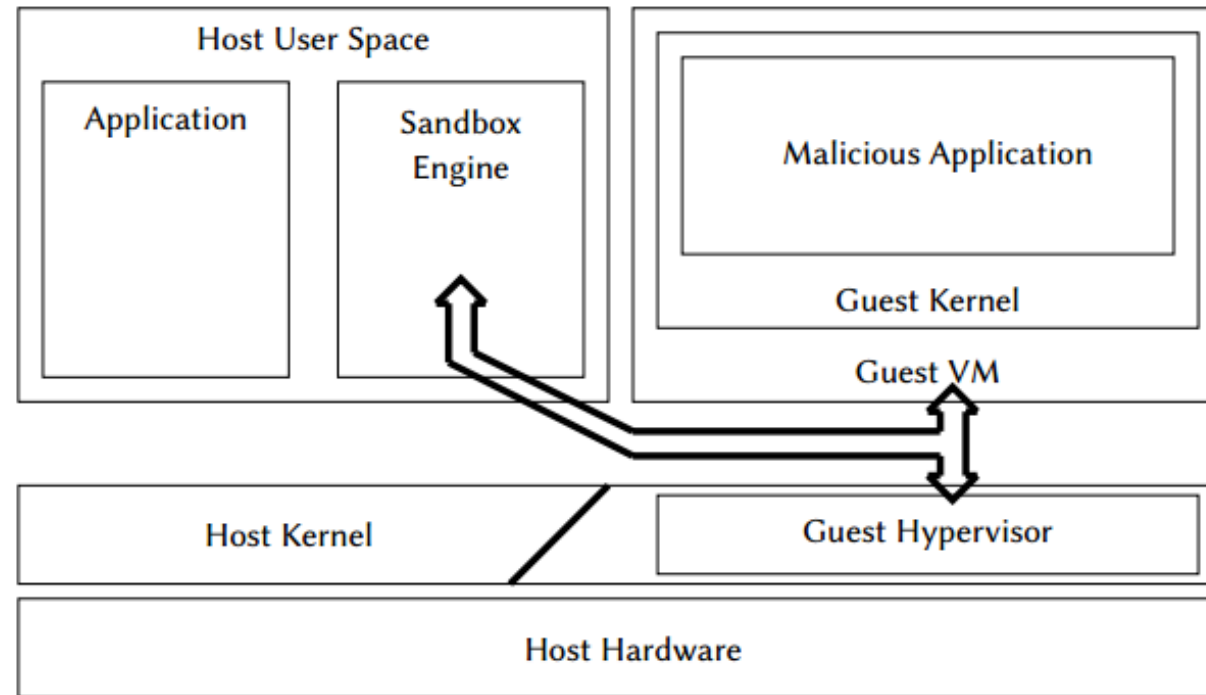
Table 1: Comparison of Dynamic Malware Analysis Approaches

System / Category	Core Method	Transparency	Performance Overhead
ANY.RUN / Cuckoo Sandbox [1, 2]	In-guest agent within full VM	Low	Moderate (VM overhead)
DRAKVUF / Ether [3, 4]	Hypervisor-based VMI	High	High (Full-system virtualization)
MALT [5]	SMM and PMUs on bare metal	Very High	Extreme (Up to 973x slowdown)
Ninja [6]	TrustZone, PMU and ETM on bare metal	Very High	Low (Instruction Trace) to High (API Trace)



Agent-based Sandbox Architecture.

Figure 2 presents the possible implementation of a sandbox that uses an agent for VM isolation. By the arrow that connects the sandbox engine directly to the sandbox agent within the guest VM, the architecture demonstrates direct interaction between the host's sandbox engine and the guest's sandbox agent.



Agent-less Sandbox Architecture.

These architectures based on VM, isolate the kernel from the host to maintain the safety.

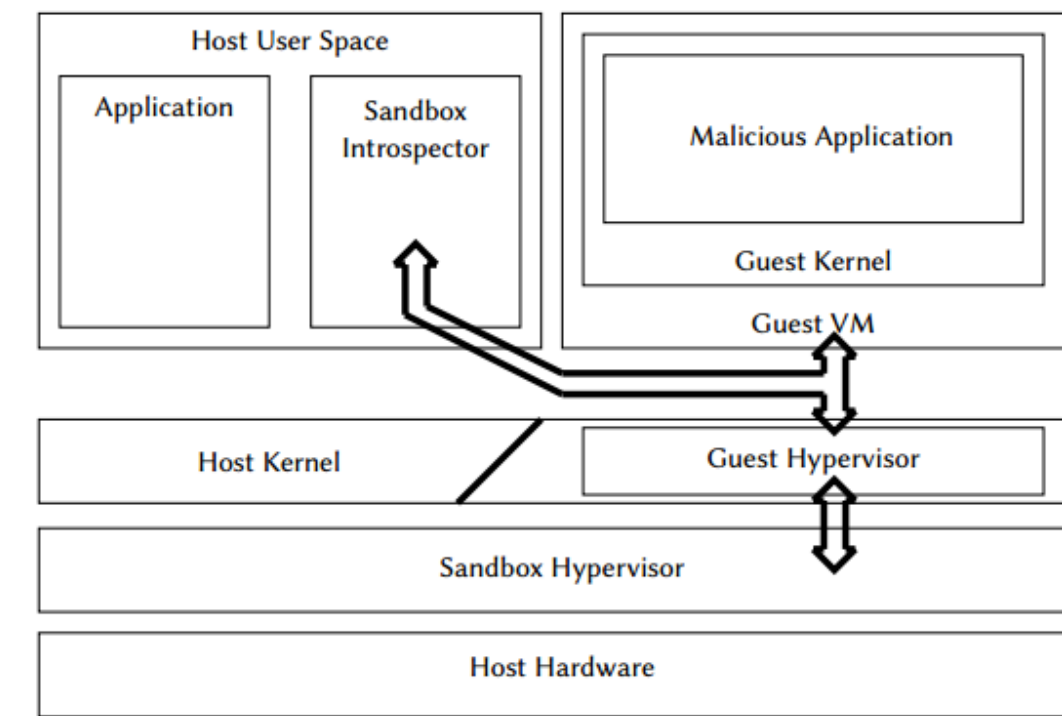


Figure 4: Hypervisor based Sandbox Architecture.

Figure 2 shows the architecture of a sandbox that takes advantage of an agent-less paradigm. As the arrows indicate, the sandbox engine interacts directly with the guest VM, via the guest hypervisor.

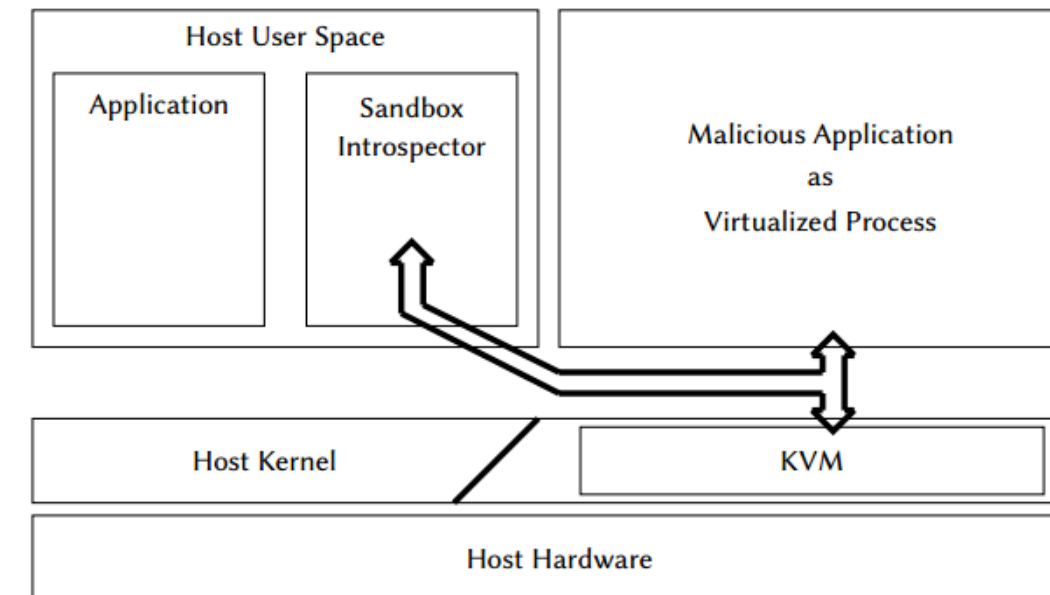
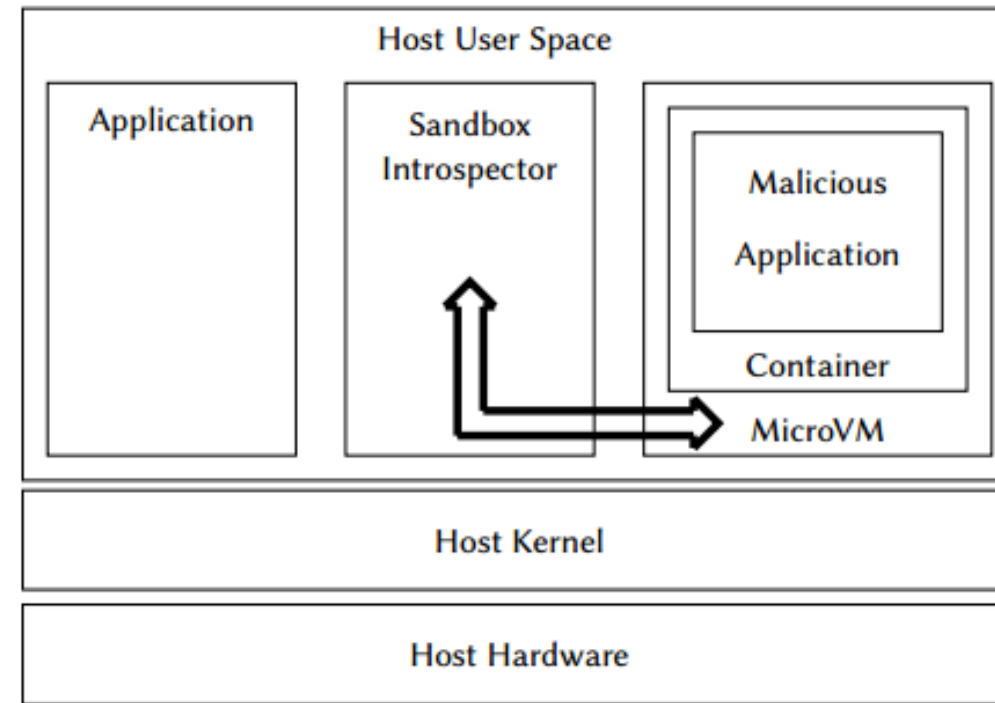


Figure 3: User Space based Sandbox Architecture.

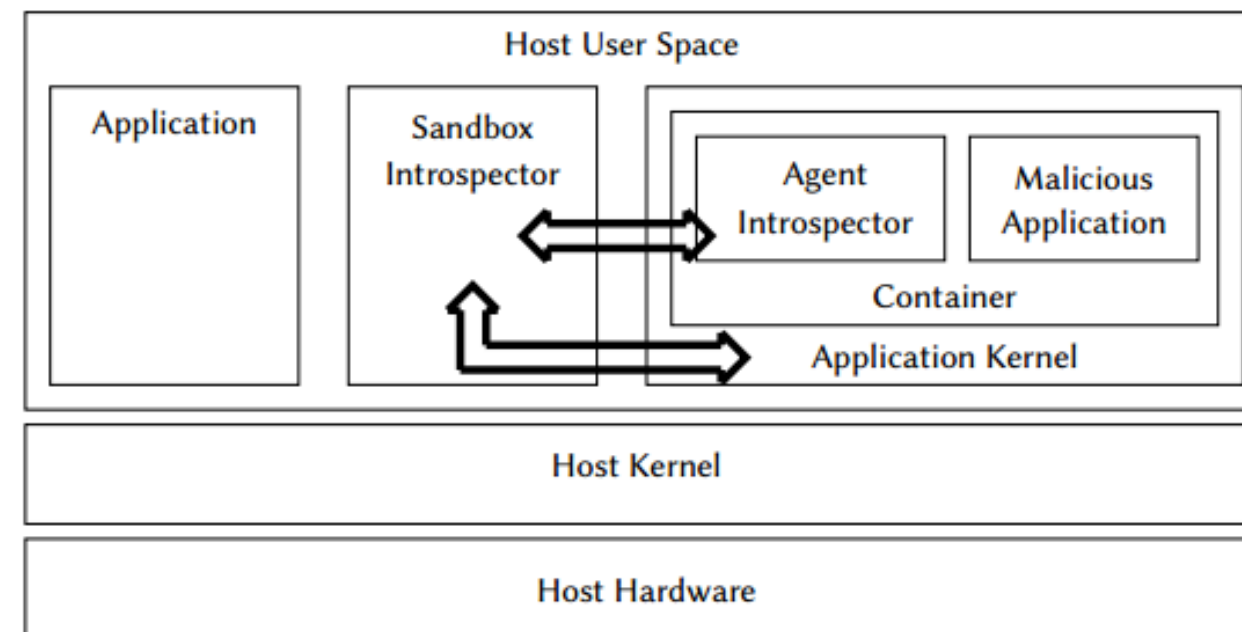


Container Sandboxing

Kata, Firecracker

Figure 5: VM based Containerized Sandboxing Architecture.

Figure 5 shows how the architecture of a containerized sandbox, like Firecracker, can be implemented. As we can see, similarly to the previous figure of the hypervisor-based sandbox, this architecture leverages the VM Introspection feature offered by the VM.



gVisor (Google)

Figure 6: Application Kernel based Containerized Sandboxing Architecture.

02

Our Design & Advantage



Our Solution: MirrorShield

A novel framework that unites lightweight emulation for speed, deep monitoring for fidelity, hardware protection for integrity, and LLM reasoning for intelligence.



Lightweight Sandbox

Leverages Docker and QEMU-user to execute multi-architecture malware on standard x86 hosts, eliminating heavy VM boot overhead.

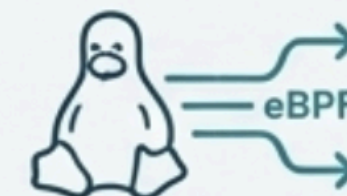


Hybrid AI Analysis

A two-stage approach overcomes LLM context limitations by using local analyzers to generate “attention hints” for the LLM.



The first practical, LLM-powered framework for *scalable, cross-architecture* Linux malware analysis.



High-Fidelity Monitoring

Uses an eBPF-based tracer for low-latency kernel monitoring, with efficient filtering and trace compression.

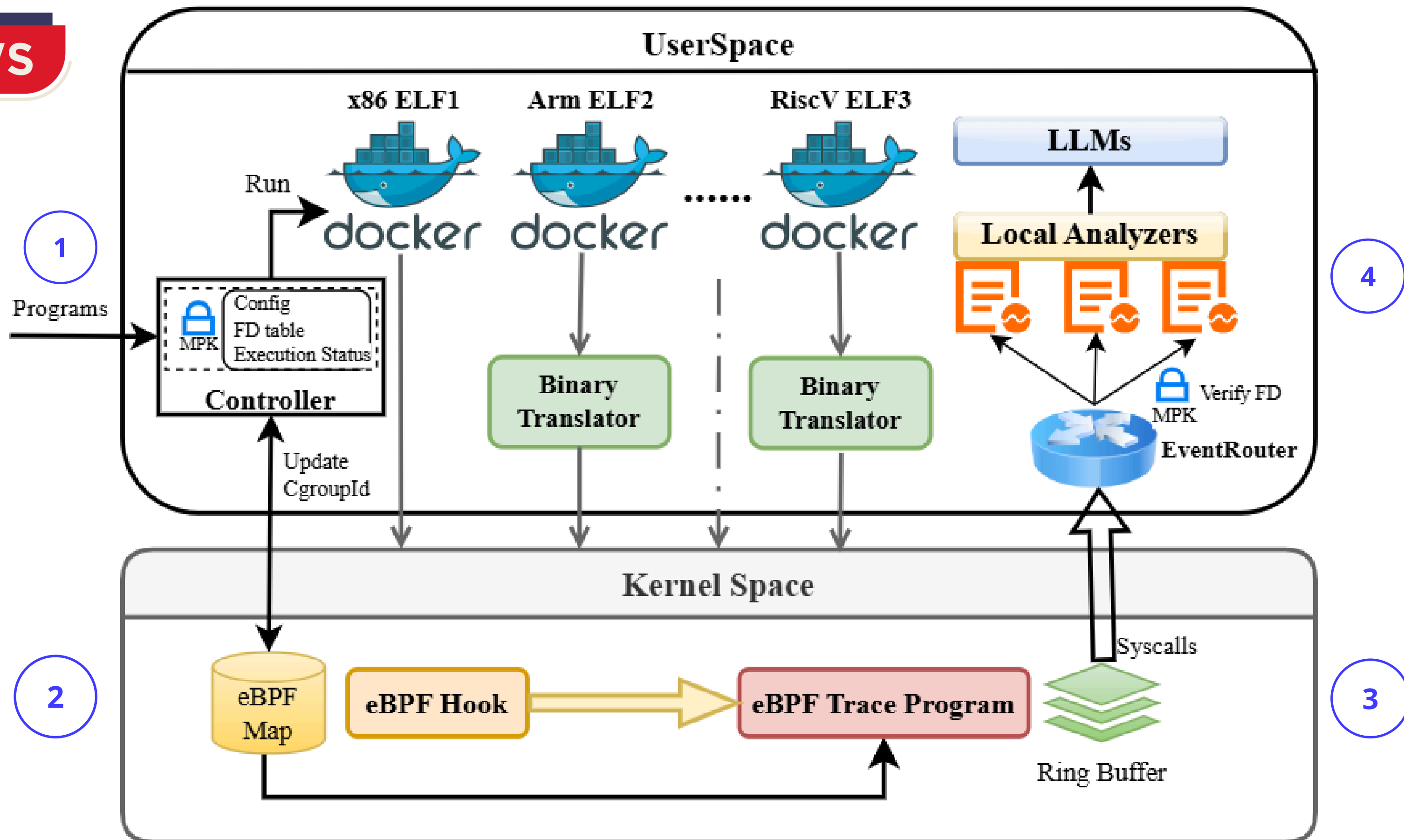


Hardware-Assisted Self-Protection

Employs Intel MPK to shield critical components of the analysis engine itself from tampering.

Workflow

NEWS



Balance Speed (Lightweight Emulation) with Depth (LLM-based Semantic Analysis).

AI-Assisted Analysis Pipeline

Raw Traces: Collected via eBPF.

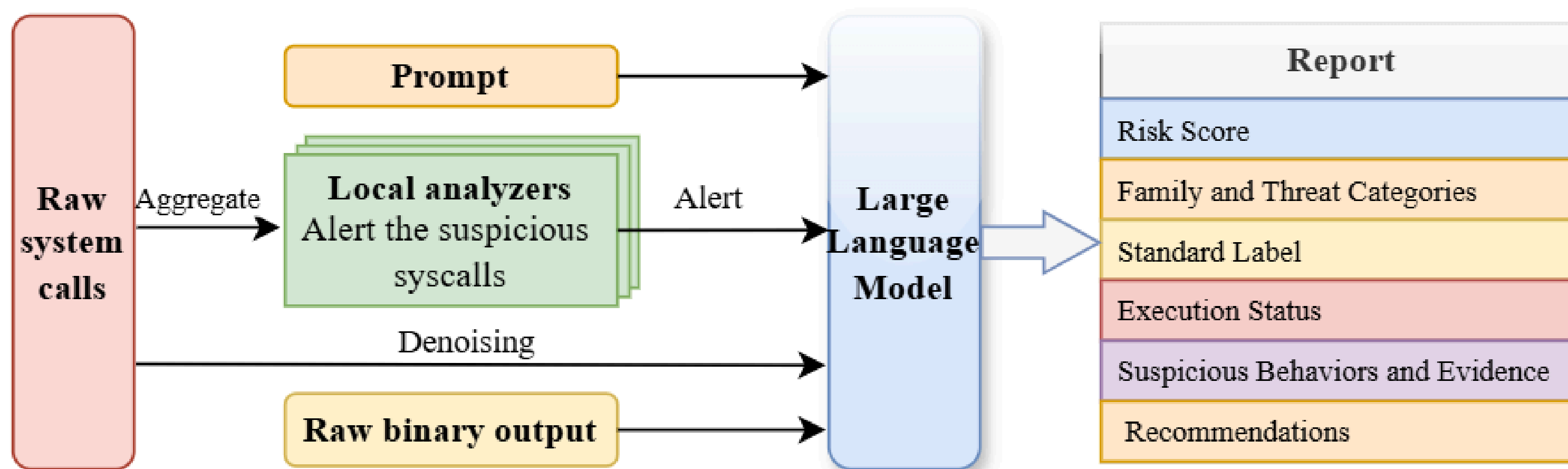
Local Analyzers : Detect known patterns (e.g.Memory Corporation, Code Injection, Container escape attempts).

Denoising: Frequency-based sampling to fit LLM context windows.

LLM Reasoning (DeepSeekV3.1):

- Contextualizes alerts.
- Reconstructs high-level intent (Dropper, DDoS, Persistence).
- Generates forensic report

A two-stage AI pipeline combines fast alerts with LLM-driven semantic reasoning to produce comprehensive reports.

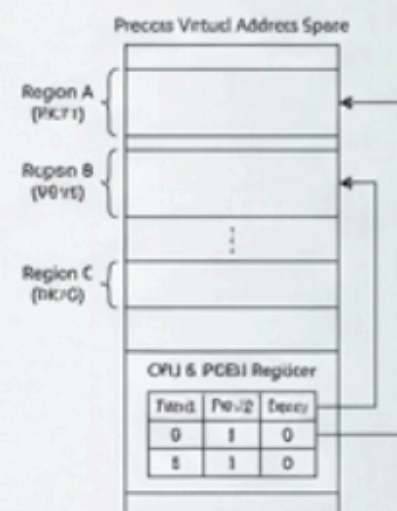


Hardware-Assisted Self-Protection

We use Intel Memory Protection Keys (MPK) to create hardware-enforced safe zones within the analysis engine itself, preventing the monitor from being attacked by the malware.

Intel MPK (PKU)

A hardware feature allowing a user process to set page-level memory permissions without costly kernel switches.



MirrorShield Controller Process

**MPK Protected:
Critical State**

Trusted Data Anchoring

Critical statistics (e.g., total/crashed sample counts) are stored in a protected memory region with a unique Protection Key. This prevents a compromised thread from fabricating success reports.

I/O Channel Integrity (FD Guard)

A protected 'shadow table' of legitimate file descriptors (FDs) prevents log hijacking attacks by cross-referencing FDs before any write operation, mitigating vulnerabilities like CVE-2024-21626.

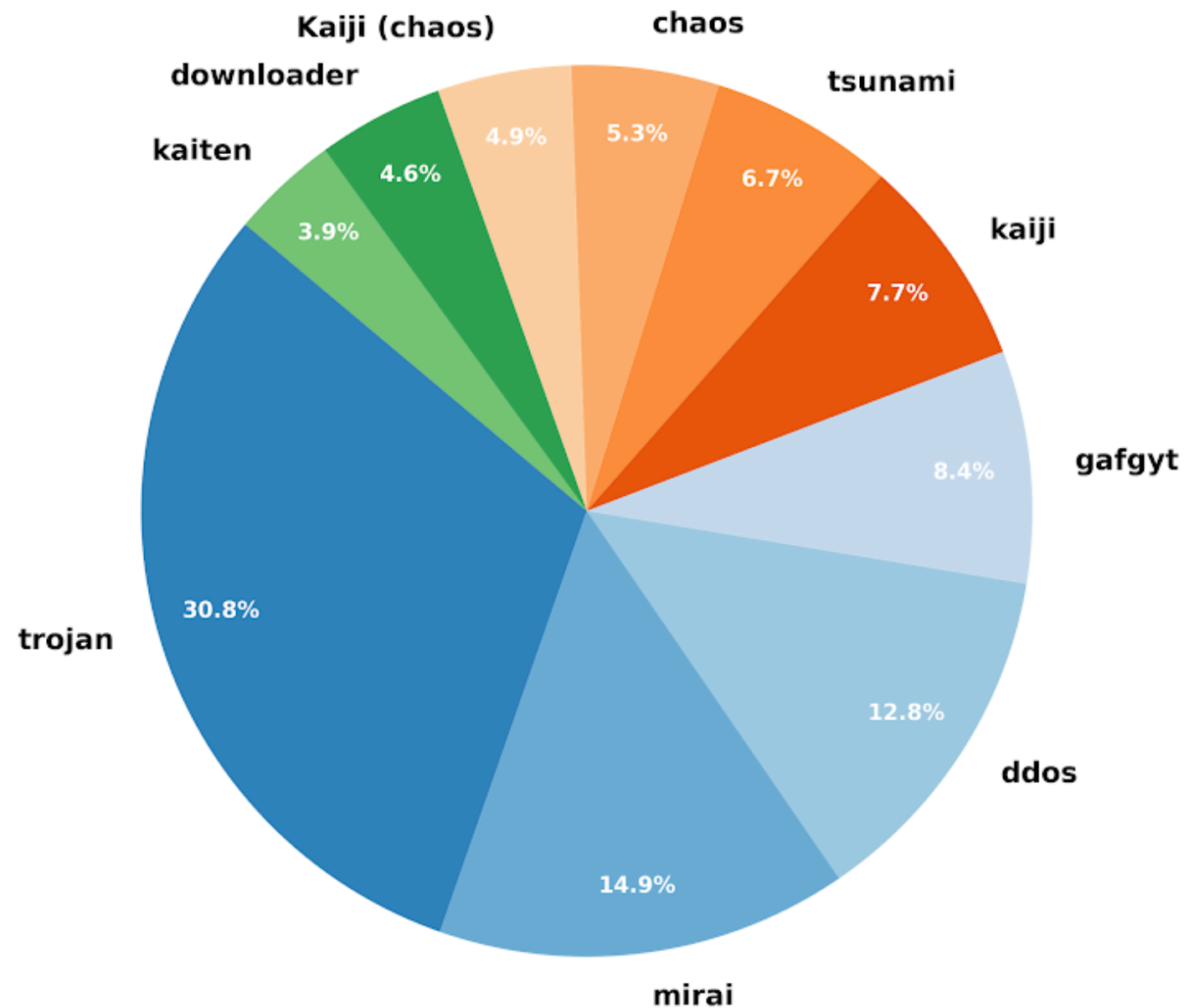
03

Evaluation

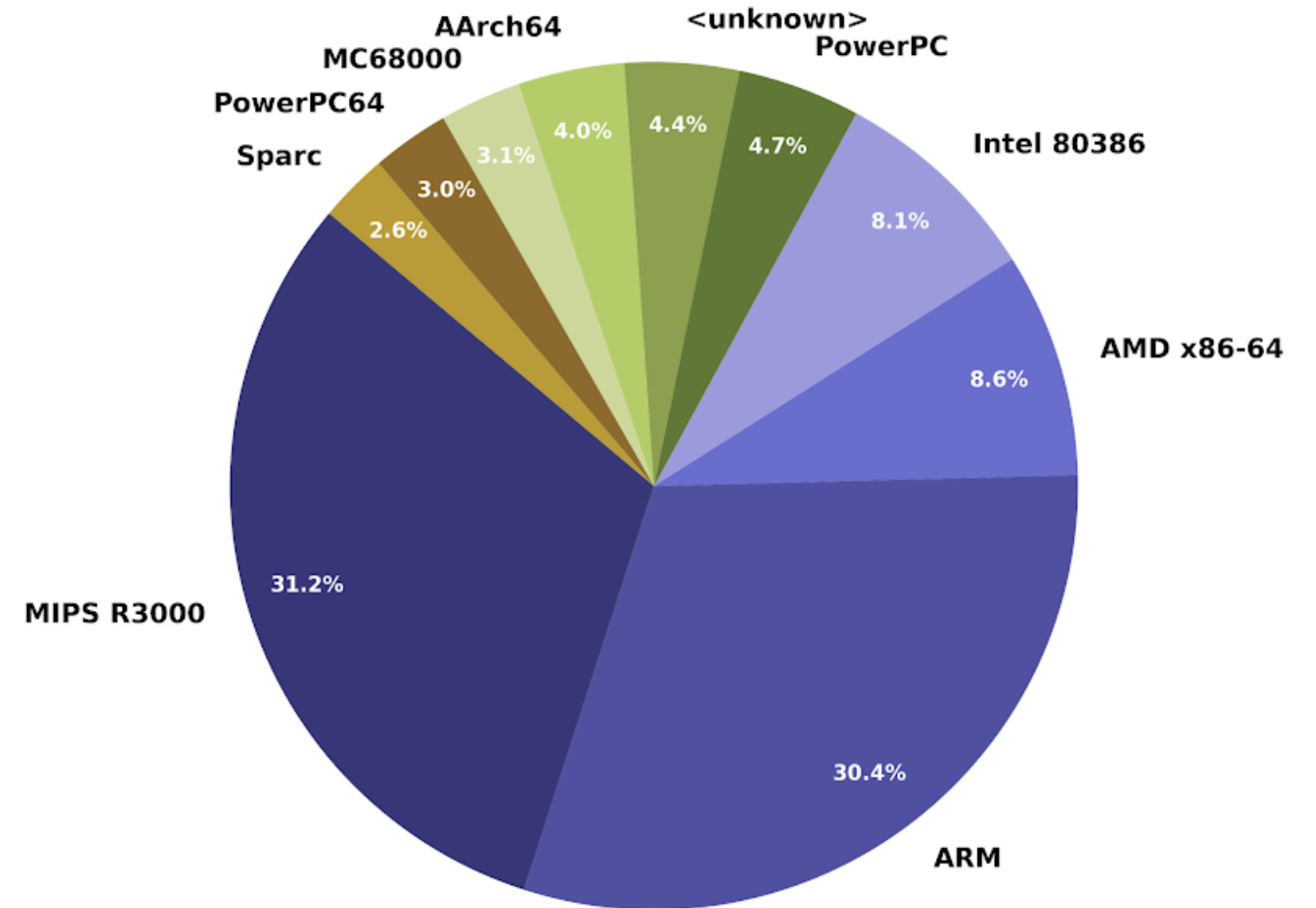


We selected 817 malware samples from <https://bazaar.abuse.ch/api/>. We also collected 133 benign ELF binaries. These samples span 7 distinct CPU architectures (including x86, ARM, MIPS, and RISC-V

Top Threat Tags Distribution



Targeted Architecture Distribution

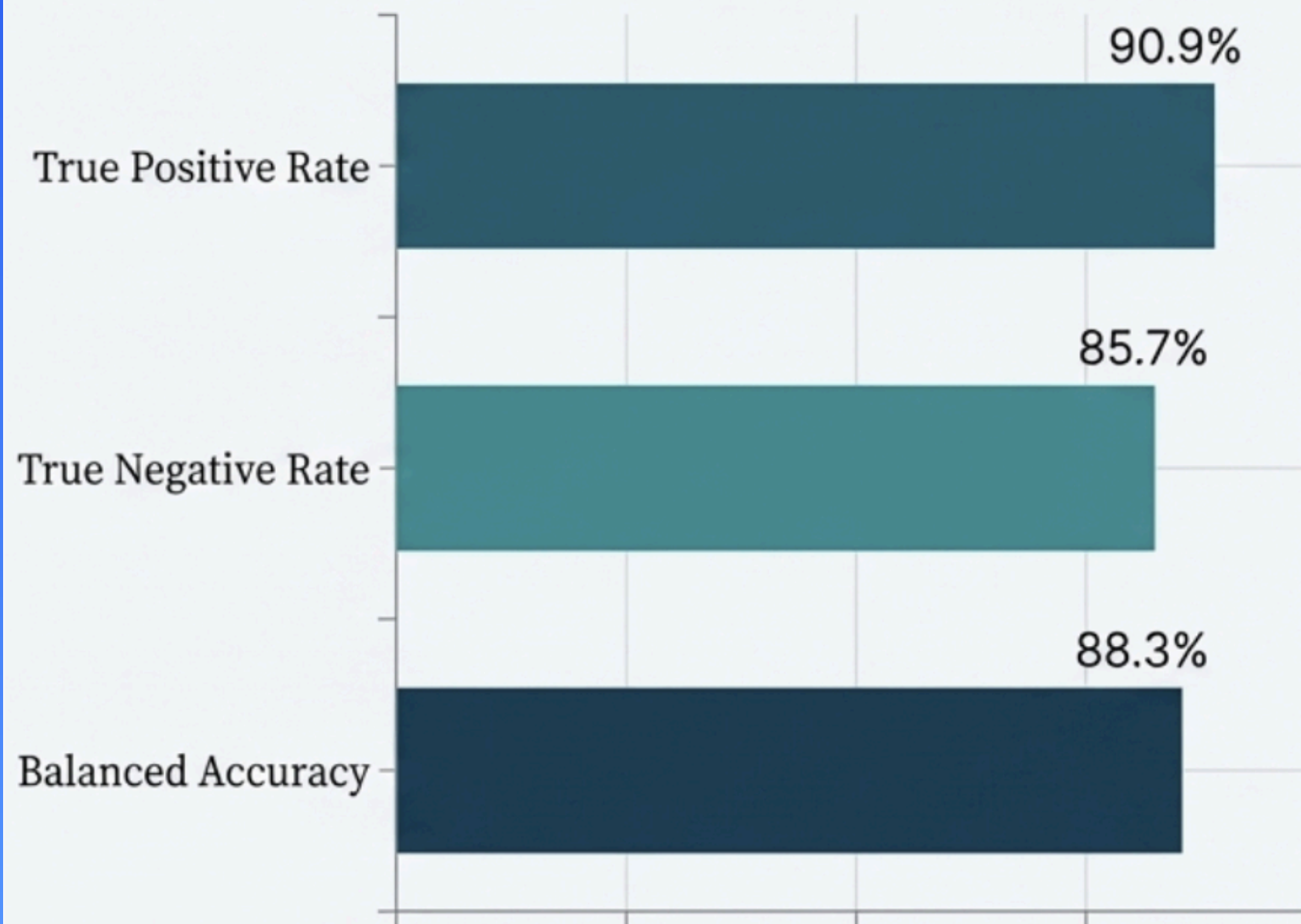


The benign dataset includes samples sourced from GitHub repositories: <https://github.com/nimrodpar/Labeled-Elfs>, <https://github.com/dur-randir/mips-binaries>, <https://github.com/cgshep/riscv-tests-prebuilt-binaries>, and <https://github.com/TheAlgorithms/C>.

Results: High Detection Accuracy and Deep Insights

MirrorShield is highly effective at identifying threats and providing meaningful, context-rich analysis that goes beyond simple detection.

Detection Performance



Semantic Analysis Performance (vs. VirusTotal)

49.9%

Threat Category Match Rate

Correctly identifies broad categories like 'DDoS' or 'Botnet'.

35.7%

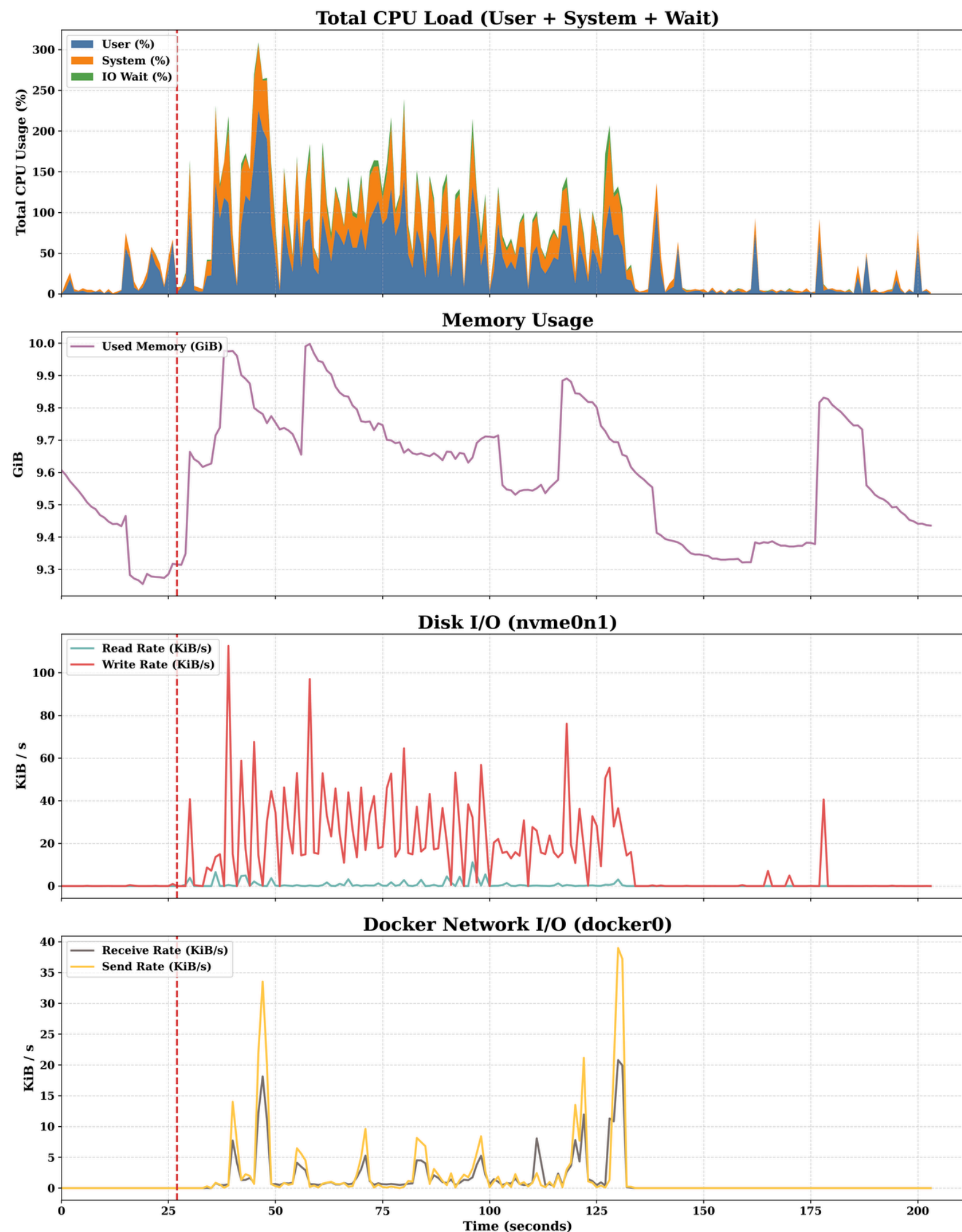
Family Match Rate

Correctly identifies specific families like 'Mirai,' showing effective characterization while acknowledging the challenge of precise family attribution.

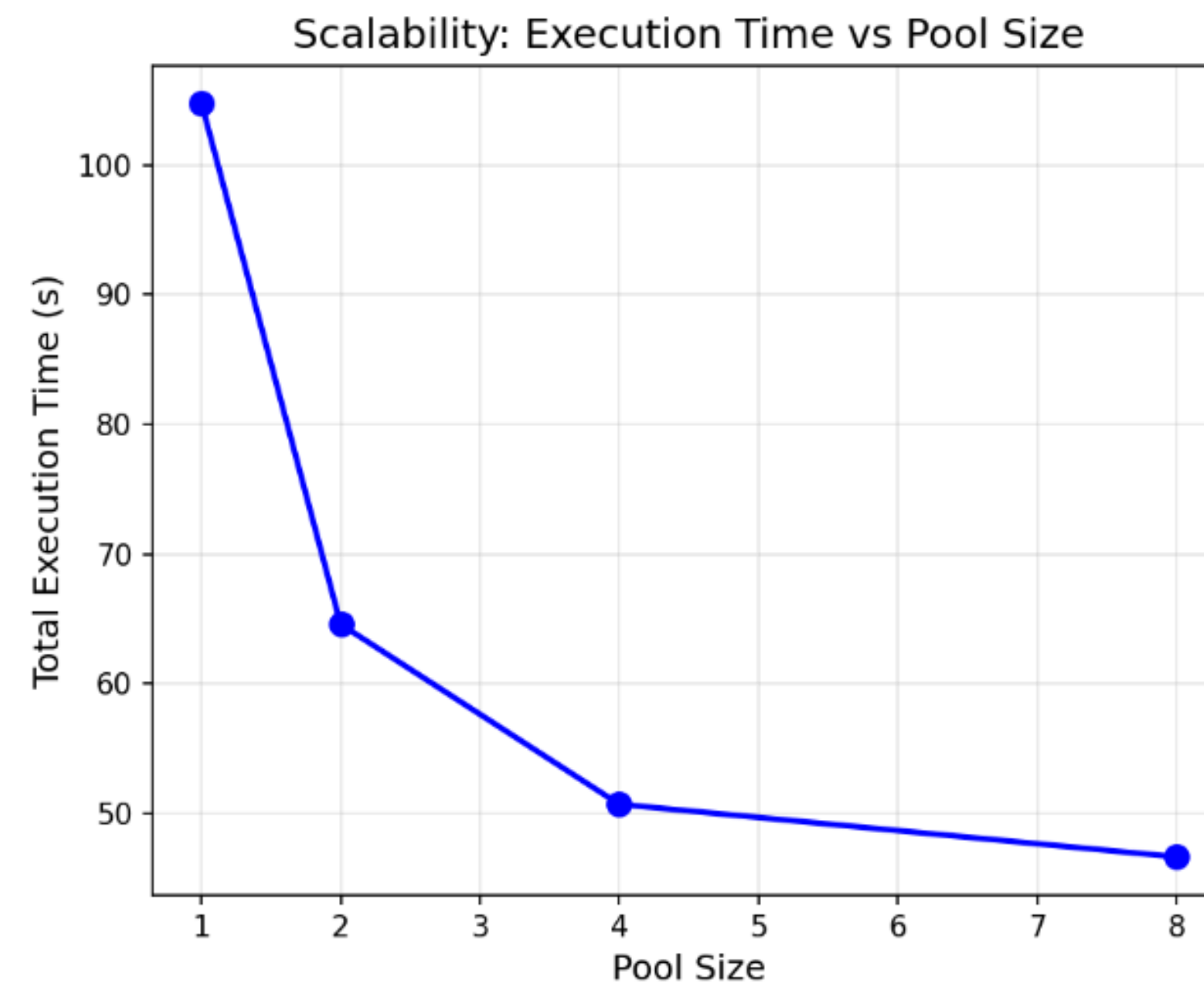
Performance

Objectives and results

Lightweight



Scalable



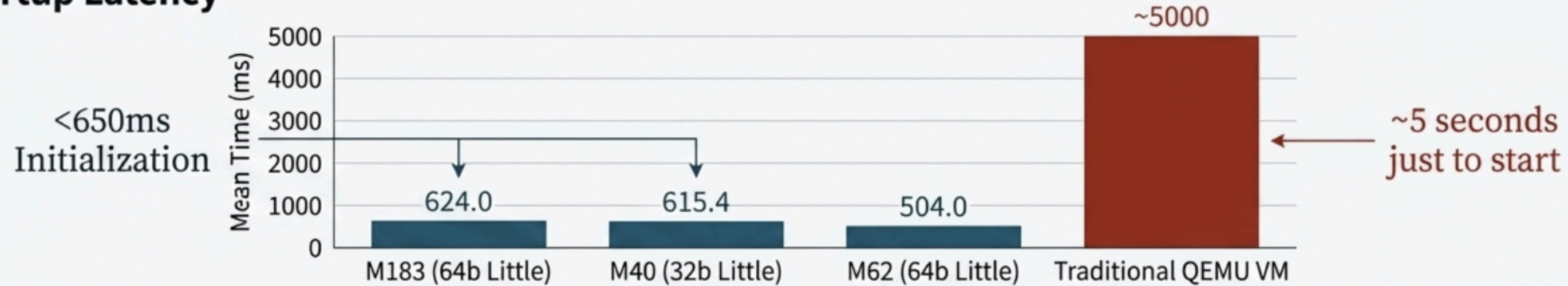
End-to-End time: **108.82 s** (483.94s) for
172 ELF 747 thousands of syscalls

Intel Core Ultra 7 258V processor, 32GB
of RAM, and a 1TB SSD. The system runs Ubuntu 25.10

Results: Orders of Magnitude Faster Than Traditional Methods

MirrorShield drastically reduces both startup and runtime overhead, enabling high-throughput analysis at a scale impossible with VM-based approaches.

Startup Latency



Runtime Overhead (System Call Latency)



Environment	Latency (μ s)	Overhead
Native Host (x86_64)	0.049	1.00×
Native strace	16.010	326.73×
MIRRORSHIELD		
x86_64	0.215	4.39×
RISC-V 64	0.326	6.65×
MIPS	0.446	9.10×
ARM64	0.472	9.63×
QEMU System VM(ARM)		
ARM64 VM + strace	891.380	18,191.4×

Case Study: Surpassing Commercial Sandboxes on a Mirai Variant

TABLE V
COMPARATIVE ANALYSIS OF SAMPLE BD7A...DA81(MIRAI VARIANT)

Commercial Sandboxes (Joe / Dr.Web)	MirrorShield
Status: Incomplete / Generic	Status: Success
1. Joe Sandbox [20]: • Result: CRASHED. • Error: fatal error: sigaction failed (Go Runtime Panic). • Verdict: Generic "Malicious" (YARA only). 2. Dr. Web vxCube [42]: • Result: High-level behavioral tags. • "Deletes a file" (No target path). • "Runs as daemon" (No context).	Attribution & Forensic Depth: • Specific Attribution: Identified as Mirai "Killer" module (surpassing generic tags). • Precise Targets: Enumerated deleted files: /etc/.ares, linux_kill. • Persistence Code: Extracted script content: while [1]; do sleep 60... • Resilience: Reconstructed attack chain despite runtime errors found in commercial tools.

LLM Analysis: Sample bd7a...da81

Final Prediction: **MALWARE** (95/100)

Family: Mirai

Status: Success

Label: Trojan.Linux.Mirai

Categories: Botnet, DDoS, Evasion

Analysis Summary: This specimen is identified as a Mirai botnet variant featuring a "Killer" module. It aggressively scans for and removes competing malware (e.g., cleaning /etc/.ares), establishes persistence via shell scripts, and attempts to manipulate system cron jobs.

opacity 0.3

Key Findings (Extracted Artifacts):

- * **Killer Behavior:** UNLINKAT events targeting competitor files: /etc/.ares, /usr/lib/libdlrpld.so, /etc/init.d/linux_kill.
- * **Persistence:** WRITE operations creating a loop script: "#!/bin/sh\nwhile [1]; do\nsleep 60...".
- * **System Tampering:** EXEC calls attempting to overwrite cron jobs (* /1 * * * * root /.mod).
- * **Process Control:** Multiple CLONE and SIGNAL_GENERATE (sig 23) events.

The LLM's reasoning provides critical forensic details that leading commercial tools miss, especially when samples are unstable or crash during execution.

04

Limitations and our future work



CVE Identifier	Summary of the Attack
CVE-2022-0492	An escape attack that works by modifying the cgroup release_agent file ¹ . Attackers exploit a feature where the kernel executes a program specified in the release_agent file when a cgroup becomes empty ² . This typically requires the container to possess the CAP_SYS_ADMIN capability to mount the cgroup and modify the file ³³³³ .
CVE-2024-21626	A runc vulnerability (affecting versions < 1.1.11) that exploits "symlink resolution deception" to bypass container isolation ⁴⁴⁴⁴ . The vulnerability stems from runc failing to close a file descriptor to the host's /sys/fs/cgroup in time when forking a child process ⁵ . This allows the container (child process) to gain access to the host's filesystem via /proc/self/fd/<fdnum> and potentially gain privileged access ⁶ .
CVE-2019-5736	The root cause of the vulnerability lies in the implementation of runC. By leveraging a specially crafted container image or exec operation, an attacker can obtain a file handle to runC on the host machine and tamper with the runC binary. This allows the execution of arbitrary commands with root privileges on the host.

Limitation

Objectives and results

Source	Approach	Architecture	Dataset	Feature	Algorithm	Accuracy
[14]	Static	ARM, MIPS, x86, x64, PowerPC, SPARC, Renesas SH	M:111,000 B:111,000	Entry Point of Code	SVM, k-NN, NB, MLP	99.96 %
[27]	Dynamic	-	M:200 B:200	System Calls	RNN	98.71 %
[29]	Dynamic	-	100,000	API, OS, File System, Miscellaneous	Clustering, Probabilistic, DL	79.6-97.4 %
[28]	Dynamic	ARM	M:1,000 B:401	System Calls, Network, VFS	CNN	99.28 %
[26]	Dynamic	MIPS	M: 928 B: 296	System Calls	LSTM	98.37 %
Ours	Dynamic	MIPS, ARM, x86	M:524 B:524	System Calls	RF, SVM, Bagging, LR, MLP*	98.57 %



Semantic Attribution Challenges

LLMs can struggle to synthesize global threat objectives from atomic actions, explaining the gap between our high detection rate (90.9%) and lower family attribution (35.7%). They often rely on superficial technical interpretation rather than reasoning through the full causal chain.



False Positives in Benign Samples

Complex but legitimate software can be misclassified due to a “confirmatory noise effect.” LLMs may misinterpret unusual but legal API usage as malicious, especially since malware often abuses legitimate APIs.

THANK YOU



Contact me if you are interested in this project

3075047037@qq.com

